

6.6 Les chaînes de caractères

Définition

- Un caractère : un chiffre, une lettre ou un symbole (A,7,; , , ...)
- Une chaîne de caractères : est une suite ordonnée de caractères, comme un mot, une phrase.
- Les guillemets : signifient à Python qu'une variable est une chaîne.

Déclaration d'une chaîne de caractères en Python

```
1 ch=str()
2 ch1="celle-ci est une chaine par des guillemets"
3 ch2=str(2022)
4 ch3='une autre chaine par un apostrophe'
5 ch4=str(' ') # une chaine contenant un espace
6 ch5=str(abc) # produit une erreur
```

Les chaînes de caractères, en tant qu'ensemble ordonné de caractères, ont beaucoup de choses en commun avec les listes :

Concaténation et duplication

on peut concaténer des chaînes en utilisant `+` ou les dupliquer par `*`

```
1 ch1="celle-ci est "
2 ch3='une chaine'
3 concatenation= ch1+ch3
4 dupliquer=ch3*3
5 print(concatenation)
6 #celle-ci est une chaine
7 print(dupliquer)
8 #une chaineune chaineune chaine
```

On peut accéder aux éléments d'une chaîne en utilisant les indices :

Accès à une chaîne

```
1 ch1="celle-ci est "
2 ch3='une chaine'
3 print(ch1[-1])
4 print(ch1[1:-3])
```

Taille d'une chaîne

```
1 ch3='une chaine'
2 print(len(ch3))
```



Attention :

Une différence majeure entre une liste et une chaîne est que la chaîne est non modifiable **on dit qu'elle est immutable**) :

Exemple

```
1 ch3='Une chaine'
2 ch3[0]="U"
3 #TypeError: 'str' object does not support item assignment
```

Les opérations `append()`, `insert()`, `pop()`, `remove()` ... ne sont pas applicables sur les chaînes !

Le symbole `\` dans une chaîne de caractères est un caractère spécial : il permet de transformer le caractère suivant en des instructions à exécuter :

- `\n` est un saut de ligne
- `\t` est une tabulation
- `\b` est un backspace
- `\a` est un bip (BEL dans la table ASCII)

- \' est un ' , mais il ne ferme pas la chaîne de caractères
- \" est un \" , mais il ne ferme pas la chaîne de caractères
- \\ est un \

Exemple

```
1 ch1="celle-ci est "
2 ch3='une chaîne'
3 print(ch1 + "\t" + ch3)
4 #celle-ci est      une chaîne
```

Fonctions utiles pour les chaînes de caractères

Appartenance d'un élément dans une chaîne

```
1 x="el"
2 if x in ch1:
3     ....
```

Décomptage str.count()

str.count(ch1,ch2) permet de compter le nombre d'occurrences de la chaîne **ch2** dans **ch1** sans recouvrement de **ch1**

```
1 ch = "ABABABAB"
2 print(str.count(ch, "A")) # en sortie 4
3 #ou bien print(ch.count("A"))
4 print(str.count(ch,"AB")) # en sortie 4
5 print(str.count(ch,"ABA")) # en sortie 2
```

Indice d'un élément

De façon similaire aux listes, on a une fonction **str.index(ch1, ch2)** (qui renvoie l'index indiquant où commence la sous-chaîne de **ch1** égale à **ch2**). La fonction renvoie une erreur si **ch2** n'existe pas :

```
1 ch1="celle-ci est "
2 ch3='le'
3 print(str.index(ch1,ch3)) # en sortie 3
```

Rassemblage en utilisant str.join()

str.join(sep, Liste) prend en argument une chaîne **sep** et une liste de chaînes **Liste** et renvoie la concaténation de tous les éléments de **Liste**, en glissant la chaîne **sep** entre chacun. **ch2** dans **ch1** sans recouvrement de **ch1**

```
1 Liste=['c','e','l','l','e','-','c','i']
2 print(str.join(', ',Liste))
3 #c,e,l,l,e,-,c,i
4 print(str.join(' ',Liste))
5 #celle-ci
```

Découpage d'une chaîne par str.split()

la fonction **str.split(ch, sep)** « découpe » une chaîne passée en argument et retourne une liste de chaînes. Par défaut, le découpage se fait sur les espaces.

```
1 ch1="celle-ci est "
2 print(ch1.split())
3 #['celle-ci', 'est']
4 print(ch1.split('e'))
5 #['c', 'll', '-ci ', 'st ']
```

suppression des caractères non imprimables `str.strip()`

Cette fonction permet de supprimer les caractères non imprimables aux extrémités des chaînes de caractères.

```
1 ch1="\n celle-ci est      \n      "  
2 print(str.strip(ch1))  
3 # 'celle-ci est'
```

La casse des chaînes

les fonctions `lower()`, `upper()`, `capitalize()` renvoient respectivement la chaîne en minuscule, majuscule et la première lettre en majuscule.

```
1 ch1="une cHaîne"  
2 print(str.upper(ch1))  
3 #UNE CHAÎNE  
4 print(ch1.lower())  
5 #une chaîne  
6 print(ch1.capitalize())  
7 #Une chaîne
```

7 Les piles et les files d'attente

Les **piles** (stack en anglais) et les **files** (queue en anglais) sont des structures abstraites de données fondamentales en informatique. Elles diffèrent par les conditions d'ajout et d'accès aux éléments qui les constituent. L'implémentation en **Python** de chacune des deux structures se fait par les **listes**.

7.1 Les PILES

Définition 2

Les piles sont des structures de données fondées sur le principe **LIFO** (**Last In, First Out**) ou "dernier arrivé, premier sorti".



Attention !

Dans une *Pile*, on n'a accès qu'au dernier élément (sommet). Pour chercher un élément donné on doit tout enlever jusqu'à ce qu'on le trouve.

La structure de données *PILE* est fortement utilisée :

- Traitements de texte : Historique des modifications (Ctrl+Z)
- Navigateurs : Historique de navigation
- Calculatrice HP 15C : Notation polonaise inversée ($1\ 2\ +\ 4\ \times\ 3\ +$)
- Algorithmes récursifs : Pile d'appels
- Vérifier les parenthésages correctes des expressions arithmétiques.
- ...

Bibliothèque des fonctions pour les PILES

Les fonctions associées aux piles sont les suivantes :

- **creerPile()** : renvoie une pile vide.
- **estVide(pile)** : renvoie *True* si la pile est vide, *False* sinon.
- **sommet(pile)** : renvoie l'élément sommet de la pile.
- **depiler(pile)** : retire le sommet de la pile.
- **empiler(elt,pile)** : ajoute au sommet de la pile l'élément *elt*.

Programme en Python

```

1 def creerPile():
2     .....
3     .....
4 def estVide(pile):
5     .....
6     .....
7     .....
8 def sommet(pile):
9     .....
10    .....
11 def empiler(pile, elt):
12    .....
13    .....
14 def depiler(pile):
15    .....
16    .....
```



Exercice

Écrire la fonction `depilerK(pile,k)` et `depilerRecurssice(pile,k)` qui permettent de dépiler les k premiers éléments de la pile. si k dépasse la taille de la pile alors la fonction dépile tous les éléments de la pile.

7.2 Les FILES

Définition 3

Les files d'attente sont des structures de données fondées sur le principe **FIFO (First In, First Out)** ou "Premier arrivé, premier sorti".



Attention !

Dans une *File*, on n'a accès qu'au premier élément arrivé.

La structure de données *FILE* peut être utilisée :

- Queue à la caisse !
- les mémoires tampons (buffers) ;
- les systèmes d'exploitation multitâches qui répartissent le temps-machine entre différentes tâches ;
- les serveurs d'impression qui traitent les requêtes dans l'ordre d'arrivée

Bibliothèque des fonctions pour les PILES

Les fonctions associées aux files sont les suivantes :

- `creerFile()` : renvoie une file vide.
- `estVide(file)` : renvoie *vrai* si la file est vide, *faux* sinon.
- `premier(file)` : renvoie le premier élément de la file.
- `defiler(file)` : retire le premier élément de la file.
- `enfiler(file,elt)` : ajoute à la fin de la file l'élément *elt*.

Programme en Python

```

1 def creerFile():
2     .....
3 def estVide(f):
4     .....
5 def premier(f):
6     .....
7     .....
8 def enfiler(f, elt):
9     .....
10    .....
11 def defiler(f):
12    .....
13    .....
```



Exercices

- Écrire la fonction `copierP(P,R)` qui permet de copier la pile P dans la pile R en utilisant seulement les fonctions associées aux piles et aux files.
- Écrire la fonction `inverserP(P)` qui permet d'inverser la pile P en utilisant les fonctions primitives des files.

8 Les dictionnaires

8.1 Introduction

Les types de données composites que nous avons abordés jusqu'à présent (listes, chaînes) étaient tous des séquences, c'est-à-dire des suites ordonnées d'éléments. Dans une séquence, il est facile d'accéder à un élément quelconque à l'aide d'un index (un nombre entier), mais à la condition expresse de connaître son emplacement.

Définition 8.1

Un dictionnaire est une structure de données qui possède un indice spécifique appelé clé. Cette clé peut être alphabétique, numérique, ou même d'un type composite

8.2 Création d'un dictionnaire

On peut créer un dictionnaire en listant toutes ses paires clé-valeur, séparées par des virgules, et en délimitant le tout avec des accolades.

Un dictionnaire vide sera donc noté `{ }`

Exemple

```
1 d = {} # d = dict()
2 d['ordinateur'] = 'computer'
3 d['souris'] = 'mouse'
4 d['clavier'] = 'keyboard'
5 print(d)

>>> {'ordinateur': 'computer', 'souris': 'mouse', 'clavier': 'keyboard'}
```

8.3 Ajouter et récupérer une valeur

Pour ajouter des valeurs à un dictionnaire il faut indiquer une clé ainsi qu'une valeur :

Exemple

```
1 d["disque"] = "disk"
2 print(d)

>>> {'ordinateur': 'computer', 'souris': 'mouse', 'clavier': 'keyboard', 'disque': 'disk'}
```

La méthode `get` vous permet de récupérer une valeur dans un dictionnaire et si la clé est introuvable, vous pouvez donner une valeur à retourner par défaut :

La méthode `get()`

```
1 print(d["souris"])
2 d.get("clav", "Inconnu")

>>> 'mouse'
>>> 'Inconnu'
```

Suppression d'une valeur

```
1 del d["souris"]
2 print(d)

>>> {'ordinateur': 'computer', 'clavier': 'keyboard', 'disque': 'disk'}
```

Vider le dictionnaire

```
1 d.clear()
2 print(d)

>>> {}
```

Suppression de la variable dictionnaire

```
1 del d
```

8.4 Les méthodes `.keys()`, `.values()` et `.items()`

La méthode `keys()` renvoie un objet itérable composé de toutes les clés utilisées dans le dictionnaire.

dictionnaire.keys()

```
1 d.keys()
>>> dict_keys(['ordinateur', 'souris', 'clavier', 'disque'])
```

La méthode `values()` renvoie un objet itérable composé de toutes les valeurs mémorisées dans le dictionnaire.

dictionnaire.values()

```
1 d.values()
>>> dict_values(['computer', 'mouse', 'keyboard', 'disk'])
```

La méthode `items()` extrait du dictionnaire un objet itérable équivalent de tuples.

dictionnaire.items()

```
1 d.items()
>>> dict_items([('ordinateur', 'computer'), ('souris', 'mouse'), ('clavier', 'keyboard'), ('disque', 'disk')])
```

8.5 Parcours d'un dictionnaire

Vous pouvez utiliser une boucle `for` pour traiter successivement tous les éléments contenus dans un dictionnaire, mais attention :

- Au cours de l'itération, ce sont les clés utilisées dans le dictionnaire qui seront successivement affectées à la variable de travail, et non les valeurs ;
- L'ordre dans lequel les éléments seront extraits est imprévisible (puisque un dictionnaire n'est pas une séquence).

Exemple de parcours d'un dictionnaire

```
1 for cle in d:
2     print(cle)

>>>
ordinateur
souris
clavier
disque
```

Si vous souhaitez effectuer un traitement sur les valeurs, il est recommandé de faire appel à la méthode `items()`.

parcours d'un dictionnaire par `items()`

```
1 for cle, val in d.items():
2     print(cle, val)

>>>
ordinateur computer
disque disk
souris mouse
clavier keyboard
```


8.6 une copie indépendante d'un dictionnaire



Attention

Vous ne pouvez pas copier un dictionnaire en faisant `dic1 = dic2!!`

Problème de copie directe d'un dictionnaire

```
1 d = {'ordinateur': 'computer', 'souris': 'mouse', 'clavier': 'keyboard'}
2 dic2 = d
3 dic2["clef"] = "key"
4 print(d)

>>> {'ordinateur': 'computer', 'souris': 'mouse', 'clavier': 'keyboard', 'clef': 'key'}
```

Pour créer une copie indépendante vous pouvez utiliser la méthode `copy()`

Création de copie indépendante

```
1 d = {'ordinateur': 'computer', 'souris': 'mouse', 'clavier': 'keyboard'}
2 dic2 = d.copy()
3 dic2["clef"] = "key"
4 print(d)

>>> {'ordinateur': 'computer', 'souris': 'mouse', 'clavier': 'keyboard'}
```

8.7 Fusionner des dictionnaires

Pour fusionner deux dictionnaires, vous pouvez utiliser la méthode `update`.

Fusion de deux dictionnaires

```
1 dic1 = {'ordinateur': 'computer', 'souris': 'mouse'}
2 dic2 = {'clavier': 'keyboard', 'clef': 'key'}
3 dic1.update(dic2)
4 print(dic1)

>>> {'ordinateur': 'computer', 'souris': 'mouse', 'clavier': 'keyboard', 'clef': 'key'}
```

9 Les tuples

Définition 9.1

Un *n-uplet* (ou tuple en anglais) est une séquence immutable (non modifiable) et ordonnée d'objets Python. Cela signifie que chaque élément du tuple est associé à une position (un index).

9.1 Déclaration d'un tuple

Déclaration

```
1 T=tuple() # déclaration d'un tuple vide
2 T1=(1,) #déclaration d'un tuple à une seule valeur
3 T2=(1,2,3,4,"tuple",10) # déclaration d'un tuple
4 T3="bonjour", 1, "tout", 2
5 a,b=(5,6) # a et b ne sont pas des tuples dans ce cas
6 print(type(a))

>>> int
```



Attention !

Pour déclarer un tuple à une seule valeur, il faut utiliser une virgule, sinon l'interpréteur va le considérer comme une expression arithmétique!

9.2 L'accès aux éléments d'un tuple

On peut accéder aux éléments d'un tuple en utilisant les indices :

Accès aux éléments d'un tuple

```
1 mon_tuple = (10, 20, 30, 40)
2 premier_element = mon_tuple[0]
3 dernier_element = mon_tuple[-1]
4 une_partie=mon_tuple[1:-1]
5 print(premier_element, type(premier_element))
6 print(dernier_element, type(dernier_element))
7 print(une_partie, type(une_partie))

>>> (10, <type 'int'>)
>>> (40, <type 'int'>)
>>> ((20, 30), <type 'tuple'>)
```

Taille d'un tuple

```
1 mon_tuple = (10, 20, "test", 30, 40)
2 print(len(mon_tuple))

>>> 5
```

On peut accéder aux éléments d'un tuple en utilisant les boucles :

Accès par boucles

```
1 mon_tuple = (10, 20, "test", 30, 40)
2 for element in mon_tuple:
3     print(element)
4 for i in range(len(mon_tuple)):
5     print(mon_tuple[i])
```

Tester l'appartenance

```
1 if x in mon_tuple:
2     #instruction à faire
```

Décomptage en utilisant count()

permet de compter le nombre d'occurrence de la variable **v** dans **mon_tuple**.

```
1 mon_tuple = (10, 20, "test", 30, 40)
2 print(mon_tuple.count(30))

>>> 1
```

Indice d'un élément

De façon similaire aux listes, on a une fonction **tuple.index(T, v)** (qui renvoie l'index de *v* dans *T*. La fonction renvoie une erreur si *v* n'existe pas :

```
1 mon_tuple = (10, 20, "test", 30, 40)
2 mon_tuple.index("test")
3 #affiche 2 comme résultat
```

Conversion d'une liste/chaîne en un tuple par le constructeur tuple()

Il prend en argument une chaîne ou une liste **ch** et renvoie un tuple formé des éléments de **ch**.

```
1 t=tuple("bonjour")
2 prin(t)

>>> ('b', 'o', 'n', 'j', 'o', 'u', 'r')
```

Concaténation des tuples

Comme pour les chaînes de caractères

```
1 T1=('b', 'o', 'n', 'j', 'o', 'u', 'r')
2 T2=('t', 'o', 'u', 't')
3 print(T1+T2)
4 print(T2*3)

>>> ('b', 'o', 'n', 'j', 'o', 'u', 'r', 't', 'o', 'u', 't')
>>> ('t', 'o', 'u', 't', 't', 'o', 'u', 't')
```

10 Les ensembles

Définition 10.1

Un ensemble est une séquence de données mutable, non ordonnée et qui ne contient pas de répétition.

Déclaration des ensembles par la méthode set()

```
1 T=set() # déclaration d'un ensemble
2 T1=set((1,)) #conversion d'un tuple en un ensemble
3 T2={1,2,3,4,"ensemble",10} # déclaration d'un ensemble
4 T3=set("MPSI44") # conversion d'une chaîne en un ensemble
```

10.1 Accès aux éléments d'un ensemble

On peut accéder aux éléments d'un ensemble en utilisant les boucles :

Accès aux éléments d'un ensemble

```
1 ens = {10, 20, "test", 30, 40}
2 for element in ens:
3     print(element)
```

Tester l'appartenance

```
1 if x in ens:
2     #instruction à faire
```

L'ajout des éléments par la fonction add()

```
1 ens={1,2,3,4,"ensemble",10}
2 ens.add(15)
3 print(ens)

>>> {1, 2, 3, 4, 'ensemble', 10, 15}
```

10.2 Les différentes fonctions appliquées sur les ensembles

Exemple sur l'ensemble E	Fonctionnement
E.add(v)	Ajoute v à E
E.pop()	supprime un élément arbitraire de E
E.clear()	Supprime tous les éléments de l'ensemble E
E2=E.copy()	affecte une copie de E à E2
E.difference(E2)	renvoie la différence en deux ensembles E et E2
E.discard(v)	Supprime un élément v de E s'il existe
E.intersection(E2)	Intersection des deux ensembles E et E2
E.isdisjoint(E2)	Renvoie True si deux ensembles ont une intersection nulle

<code>E2.issubset(E)</code>	renvoie <i>True</i> si <i>E2</i> est sous-ensemble de <i>E</i>
<code>E2.issuperset(E)</code>	renvoie <i>True</i> si <i>E</i> est sous-ensemble de <i>E2</i>
<code>all(E)</code>	renvoie <i>True</i> si tous les éléments de l'ensemble sont vrais (ou si l'ensemble est vide).
<code>any(E)</code>	renvoie <i>True</i> si un élément de l'ensemble est vrai. Si l'ensemble est vide, renvoie <i>False</i> .
<code>enumerate(E)</code>	Renvoie un objet énumérer. Il contient l'indice et la valeur de tous les éléments de l'ensemble sous forme de paire.
<code>len(E)</code>	la taille d'un ensemble
<code>max(E)</code>	le maximum d'un ensemble
<code>min(E)</code>	le minimum d'un ensemble
<code>sorted(E)</code>	Renvoie une nouvelle liste triée à partir des éléments de l'ensemble.
<code>sum(E)</code>	Somme de tous les éléments.

11 Manipulation des fichiers en Python

11.1 Introduction

Un programme s'exécute dans la mémoire volatile (RAM) de l'ordinateur. Pour conserver les données lors de l'exécution, il faut utiliser une mémoire permanente comme un disque ou une clé où les données sont organisées en des fichiers par le système d'exploitation.

Définition 11.1

Un fichier est une collection d'informations stockées sur une mémoire de masse non volatile, de capacité plus importante que la mémoire vive. on distingue généralement deux types de fichiers :

- Fichiers de données : des fichiers là où les données peuvent être lues par un être humain.
- Fichiers binaires : des fichiers qui peuvent être interprétés par la machine (fichier .exe ou .jpg)

11.2 Principales manipulations sur les fichiers

Les principales manipulations sur les fichiers

- En lecture (mode « **r** » comme read) : lecture seule des données contenues dans le fichier. Si le fichier n'existe pas, un message d'erreur va apparaître.
- En écriture (mode « **w** » comme write) : écriture de données dans un nouveau fichier. Si le fichier qui doit être créé existe déjà, il est écrasé par le nouveau fichier.
- En ajout (mode « **a** » comme append) : ajout de données à un fichier déjà existant. Si le fichier à ouvrir existe déjà, l'écriture des données commence à la fin du fichier. Si le fichier n'existe pas, il est créé.
- Si les opérations précédentes sont appliquées sur un fichier binaire, on doit donc ajouter **b** pour indiquer qu'il s'agit d'un fichier binaire.

11.3 Ouverture d'un fichier

Chaque fichier est associé à une sorte d'adresse qui permet de le trouver facilement sans erreur. Et Il n'est pas possible de nommer identiquement un fichier dans un même dossier (excepté si l'extension est différente). Pour ouvrir un fichier en Python, on utilise la fonction `open()` en indiquant le chemin vers le fichier à ouvrir et son mode d'ouverture. Généralement, il existe deux types de chemin : le **chemin absolu** qui part de la racine de votre système de fichier (disque C ou disque D) et le **chemin relatif** qui part du dossier en cours de lecture.

Exemple d'ouverture d'un fichier

```
1 variable=open("chemin_du_fichier/nom_fichier.extension","mode")
```

Exemple d'ouverture d'un fichier texte en mode lecture

```
1 fic=open("C:/monfichier.txt","r") # chemin absolu: le fichier à ouvrir est dans le disque C.
```

Si le fichier à manipuler existe dans le dossier courant (le dossier qui contient le fichier Python) donc on écrit le nom du fichier à ouvrir sans chemin.

```
1 fic2=open("monfichier.txt","w") # chemin relatif: le fichier à ouvrir est dans le dossier courant
```

La fonction `open()` renvoie un objet de type *fichier* stocké dans `fic`. la variable `fic` garantit alors la connexion au fichier en question.

11.4 Fermeture d'un fichier

Une fois la manipulation sur le fichier est terminée, il est nécessaire de le fermer pour interdire tout accès à ce fichier en utilisant la fonction `close()`.

Exemple d'ouverture et de fermeture de fichier

```

1 fic2=open("monfichier.txt","w") # chemin relatif: le fichier à ouvrir est dans le dossier
  courant
2 .....#code à écrire
3 fic2.close() # Verrouiller le fichier

```

11.5 Les modes de lecture des fichiers

Il existe plusieurs fonctions permettant de lire des données dans un fichier :

- `read()` : Lit toutes les données du fichier et les place dans une seule chaîne de caractères.
- `readline()` : Permet de lire une ligne dans un fichier. Si la fin du fichier est atteinte, `readline()` renvoie une chaîne vide.
- `readlines()` : Lit toutes les lignes du fichier et les place dans une liste de chaînes de caractères.
- `read(n)` : Lit n caractères et les place dans une chaîne de caractères

Exemple de `read()`

```

1 fic2=open("exception.py","r") # le fichier à ouvrir est dans le dossier courant
2 chaine=fic2.read()
3 print(chaine)
4 fic2.close() # Verrouiller le fichier

```



Résultat

```

def terme(n,a):
    s=0
    for i in range(n):
        s+=1/(a**i)
    return s
def appeler(n,a):
    return terme(n,a)

```

Exemple de `read(n)`

```

1 fic2=open("exception.py","r") # le fichier à ouvrir est dans le dossier courant
2 chaine=fic2.read(50)
3 print(chaine)
4 fic2.close() # Verrouiller le fichier

```



Résultat

```

def terme(n,a):
    s=0
    for i in range(n):

```

Exemple de `readlines()`

```

1 fic2=open("exception.py","r")
2 chaine=fic2.readlines()
3 print(chaine)
4 fic2.close()

```



Résultat

```
[ 'def terme(n,a):\n', '    s=0\n', '    for i in range(n):\n', '        s+=1/(a**i)\n', '    return s\n', 'def appeler\n(n,a):\n', '    return terme(n,a)\n', '\n']
```

11.6 Les modes d'écriture dans un fichier

Il existe deux méthodes pour écrire dans un fichier en Python, la fonction `write(chaine)` et la fonction `writelines()`.

La fonction `write()`

La fonction `write ( texte )` permet d'écrire la chaîne de caractère *texte* dans un fichier.

```
1 fic2=open("exception2.py","w")
2 fic2.write("bonjour tout le monde")
3 fic2.close() # Verrouiller le fichier
```

La fonction `writelines()`

La fonction `writelines ()` est utilisée pour écrire une séquence de chaînes dans le fichier.

```
1 T=["bonjour\n","tout\n","le monde"]
2 fic2=open("exception2.py","w")
3 fic2.writelines(T)
4 fic2.close() # Verrouiller le fichier
```

11.7 Ouverture/Fermeture d'un fichier par la clause `with`

L'instruction `with` permet d'écrire un code plus court sans utiliser l'instruction `close`.

`with`

```
1 f = open("mon_fichier.txt", "r")
2 for ligne in f:
3     print(ligne)
4 f.close()
5 #----Ces deux codes sont identiques -----
6 with open("mon_fichier.txt", "r") as f:
7     for ligne in f:
8         print(ligne)
```

11.8 Exercices

1. Écrire un script qui permet de stocker dans un fichier les carrés des nombres entre 0 et 20.
2. Écrire un script qui permet de lire ces nombres à partir du fichier et de n'afficher que les nombres paires.